Primer entregable – Parte 1

Estructura del proyecto del Agente: tecnologías, modelos de base de datos y modelos de IA

Proyecto: Mike API – refrigerador inteligente con agente de IA (ESP32 M5)

Alumno: _____

Materia: _____

Profesor: Miguel Ángel Cardona Contreras

Repositorio: https://github.com/Elihu-MR/MiniBasado

Rama del entregable: v01

Fecha: 26/10/2026

Introducción

MiniBasado es el repositorio donde se documenta y desarrolla un agente de IA para un refrigerador inteligente basado en un ESP32 M5. El backend, llamado **Mike API**, administra usuarios, refrigeradores, alimentos, tickets de compra, sensores y cámaras, y expone esa información a un agente de IA que puede consultarla y modificarla.

Este primer entregable documenta, explica y justifica la base del proyecto: las tecnologías elegidas, los modelos de base de datos y los modelos de IA. Se enfoca en los cuatro componentes que sostienen todo lo demás:

- **Modelo User:** la entidad central a la que pertenecen refrigeradores, tickets y conversaciones con la IA.
- **Hasheo:** almacenamiento seguro de contraseñas con Argon2.
- **Token:** autenticación sin estado con JWT.
- **MinIO:** almacenamiento de imágenes (fotos de perfil, tickets y alimentos).

El código se encuentra en https://github.com/Elihu-MR/MiniBasado, en la rama **v01**.

Desarrollo

1. Tecnologías y justificación

Tecnología	Uso en el proyecto	Por qué se eligió
Python 3.14	Lenguaje del backend y del agente	Ecosistema fuerte en APIs e IA; mismo lenguaje para todo el proyecto
FastAPI	Framework de la API REST	Rápido, asíncrono, valida datos con Pydantic y genera documentación automática en /docs
SQLModel / SQLAlchemy	ORM y definición de modelos	Un solo modelo sirve como tabla y como esquema de validación
PostgreSQL 15	Base de datos relacional	Robusta, soporta llaves foráneas, índices únicos, enums y JSON
Alembic	Migraciones de la base de datos	Versiona los cambios del esquema sin perder datos
Argon2 (passlib)	Hasheo de contraseñas	Algoritmo ganador de la Password Hashing Competition; resistente a fuerza bruta con GPU
JWT (python-jose)	Tokens de acceso	Autenticación sin estado, ideal para clientes como apps y el ESP32
MinIO + boto3	Almacenamiento de imágenes	Compatible con Amazon S3, se ejecuta localmente y separa los archivos de la base de datos
Docker Compose	Infraestructura	Levanta PostgreSQL y MinIO con un solo comando y de forma reproducible
FastMCP	Servidor de herramientas para el agente	Expone la API a la IA mediante el protocolo estándar MCP

2. Estructura del proyecto

Carpeta / archivo	Contenido
main.py	Crea la app FastAPI, configura CORS y monta las rutas en /api/v1
core/	config.py (variables de .env), security.py (hasheo y JWT), cors.py
db/	database.py: motor de conexión y sesión de base de datos
models/	Modelos SQLModel (tablas de la base de datos)
schemas/	Esquemas Pydantic de entrada y salida (UserCreate, UserResponse, Token…)
api/v1/	Endpoints por recurso y dependencies.py (usuario actual y cliente de MinIO)
alembic/	Migraciones del esquema de la base de datos
mcp_server/	Servidor FastMCP que conecta el agente de IA con la API
docker-compose.yml	Servicios de PostgreSQL y MinIO
.env	Credenciales y configuración (no se sube al repositorio)

3. Modelos de base de datos

Modelo User (`models/user_model.py`, tabla `users`):

Campo	Tipo	Detalle
id	int	Llave primaria
username	str (3–50)	Único e indexado; se usa para iniciar sesión
email	str	Único e indexado; validado como EmailStr en el esquema
hashed_password	str	Hash Argon2; nunca se guarda la contraseña en texto plano
is_active	bool	Permite desactivar cuentas sin borrarlas
created_at	datetime	Fecha de registro
profile_pic	str (255), opcional	Nombre del objeto de la foto de perfil en MinIO

Esquemas relacionados: `UserCreate` pide username, email y password (8 a 72 caracteres); `UserResponse` devuelve los datos públicos **sin la contraseña**. La migración `039ab0175ea6` consolidó la tabla vieja `usuario` en `users`, agregando username, hashed_password, is_active y profile_pic sin perder datos.

Otros modelos del dominio (todos dependen directa o indirectamente de User):

Modelo (tabla)	Propósito
Refrigerator (refrigerators)	Refrigerador del usuario y su dispositivo ESP32
Food (foods)	Alimentos con cantidad, caducidad y estado (fresco, por caducar, caducado)
PurchaseTicket (purchase_tickets)	Tickets de compra y la URL de su imagen
FoodImage (food_images)	Imágenes de alimentos o tickets, texto OCR y análisis de IA
Sensor / SensorReading	Sensores del refrigerador (temperatura, humedad, peso, puerta) y sus lecturas
Camera / CameraEvent	Cámaras y eventos detectados (movimiento, apertura)

Location, EmergencyContact	Ubicación del usuario y contactos de emergencia
AIChatLog (ai_chat_logs)	Historial de conversación con la IA y acciones que ejecutó

4. Hasheo de contraseñas

Las contraseñas se procesan en `core/security.py` con **Argon2** mediante `passlib`:

```
pwd_context = CryptContext(
    schemes=["argon2"], deprecated="auto",
    argon2__time_cost=2, argon2__memory_cost=10240, argon2__parallelism=2,
)
def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)
def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

- Al registrarse (`POST /api/v1/`), la contraseña se convierte en hash antes de guardarse.
- Al iniciar sesión, `verify_password` compara la contraseña escrita contra el hash guardado.
- **Justificación:** Argon2 agrega una sal aleatoria y consume memoria a propósito (`memory_cost`), lo que hace muy costosos los ataques de fuerza bruta; además es el estándar recomendado por OWASP.

5. Token de acceso (JWT)

La autenticación usa **JSON Web Tokens** firmados con **HS256** y una `SECRET_KEY` definida en `.env`. Por defecto, los tokens expiran a los **30 minutos** (`ACCESS_TOKEN_EXPIRE_MINUTES`).

- **Login:** `POST /api/v1/token` recibe username y password (formulario OAuth2). Si son correctos, devuelve `{"access_token": "...", "token_type": "bearer"}`.
- **Uso:** el cliente envía el encabezado `Authorization: Bearer <token>` en cada petición protegida.
- **Validación:** la dependencia `get_current_user` decodifica el token, lee el usuario del campo `sub` y lo busca en la base de datos; si el token es inválido o expiró, responde 401.
- **Justificación:** el servidor no guarda sesiones, así que escala bien y funciona igual para una app, para el ESP32 y para el agente de IA.

[Captura sugerida: Swagger (/docs) con el login en /api/v1/token y el botón Authorize]

6. MinIO (almacenamiento de imágenes)

MinIO corre en Docker (puerto 9000 para la API S3 y 9001 para la consola web) y guarda los archivos en el bucket `fastapi-images`. La base de datos solo almacena el nombre o la URL de cada archivo.

- `get_minio_client` (en `api/v1/dependencies.py`) crea un cliente `boto3` con el endpoint y las credenciales del `.env`.
- `ensure_bucket_exists` crea el bucket si todavía no existe.
- **Foto de perfil:** `PATCH /api/v1/me/profile-pic` (requiere token) valida que sea JPEG, PNG o WebP, la sube con un nombre único (`profile_<id>_<uuid>`) y guarda ese nombre en `User.profile_pic`.
- Los tickets y fotos de alimentos que procesa el agente se guardan en las carpetas `tickets/` y `foods/` del mismo bucket.

- **Justificación:** separar los archivos de la base de datos la mantiene ligera, y al ser compatible con S3 se puede migrar a la nube sin cambiar el código.

[Captura sugerida: consola de MinIO mostrando el bucket fastapi-images]

7. Modelos de IA

El agente del proyecto usa **Claude**, un modelo de lenguaje de Anthropic con capacidad de visión (puede leer imágenes) y de uso de herramientas.

- **Cómo se conecta:** mediante un servidor **FastMCP** (`mcp_server/server.py`) que expone la API como herramientas: registrar tickets, alimentos e imágenes, consultar el refrigerador, etc. El agente nunca accede directamente a la base de datos ni a MinIO; el servidor inicia sesión con JWT y aplica las reglas.
- **Qué hace:** lee la foto de un ticket, extrae tienda, fecha y productos, y los registra en la API con sus imágenes en MinIO.
- **Dónde queda registro:** la tabla `ai_chat_logs` guarda el rol (user, assistant o system), el mensaje, la imagen adjunta, la acción que ejecutó la IA (JSON) y los tokens consumidos.
- **Justificación:** un modelo con visión permite usar fotos reales (tickets, alimentos) como entrada, y el protocolo MCP mantiene separadas la inteligencia del modelo y la seguridad de la API.

8. Repositorio en GitHub

Elemento	Valor
Repositorio	https://github.com/Elihu-MR/MiniBasado
Rama del entregable	v01
Colaborador agregado	MikeCardona076 (Miguel Ángel Cardona)
Archivos excluidos	.env, entorno virtual, minio_data y __pycache__

Comandos para publicar la rama v01:

```
git checkout -b v01
git add .
git commit -m "Primer entregable: User, hasheo, token y MinIO"
git push -u origin v01
```

[Captura sugerida: la rama v01 en GitHub y la lista de colaboradores]

Conclusión

La primera parte del proyecto deja lista la base sobre la que trabajará el agente. El **modelo User** concentra la identidad y la propiedad de los datos; el **hasheo con Argon2** protege las contraseñas incluso si la base de datos se filtra; los **tokens JWT** permiten que cualquier cliente (app, ESP32 o agente de IA) se autentique sin que el servidor guarde sesiones; y **MinIO** resuelve el almacenamiento de imágenes de forma escalable y compatible con S3.

La elección de FastAPI, SQLModel y PostgreSQL ofrece validación de datos, documentación automática y un esquema versionado con Alembic. Por su parte, conectar Claude mediante un servidor MCP permite que la IA actúe sobre la API de forma controlada, usando solo las herramientas que se le exponen.

Los siguientes pasos son ampliar los endpoints del agente, registrar sus conversaciones en `ai_chat_logs` e integrar los sensores y la cámara del ESP32 M5.